ISL SIGN RECOGNITION

Complete codebase reference

Architecture • modules • functions • interfaces • desktop UI • data • training • inference • evaluation • launch • tests

Prepared for Advik | Repository: isl-linux | Reference date: 13 October 2026

What this document covers

This is a source-oriented guide to the current repository. It maps each tracked Python module, its public or meaningful internal functions, the main data structures passed between layers, the user interface, command-line entry points, configuration, model artifacts, and the tests. It describes behavior visible in the source and repository state; it does not claim that every optional workflow has been run on every machine.

The application is a Linux desktop prototype for recognizing static Indian Sign Language alphabet fingerspelling from a webcam. The live model expects MediaPipe hand landmarks and emits letters A–Z. Text assembly is handled separately from recognition. Data download, validation, splitting, feature extraction, training, evaluation, image inference, and webcam inference are separate commands and code paths.

Reading guide

Part	Topics
1–3	Project scope, architecture, repository map, end-to-end flows
4–7	Data lifecycle, landmark representation, models, training and checkpoint contract
8–10	Inference, webcam pipeline, UI controls and state
11–14	Module-by-module function reference, commands, configuration, tests
15–17	Artifacts, known issues and careful extension guidance

Scope note: “everything” here means a detailed explanation of the files and behavior in this repository. It does not reproduce the full source text, private dataset contents, external library internals, operating-system behavior, or undocumented assumptions in upstream datasets.

1. System overview

The system has two recognition routes. The current primary route is a landmark MLP: MediaPipe detects up to two hands, the project converts each detected hand to a fixed 134-value vector, and a small PyTorch multilayer perceptron scores 26 letters. The application then applies a hand-count compatibility mask and temporal confirmation before presenting a stable letter. An older static-image CNN remains in the repository for legacy checkpoints and image preprocessing compatibility; the native live UI rejects a checkpoint whose architecture is not the landmark MLP.

Layer	Responsibility	Representative modules
User entry	Select camera, start app or training	launch-isl-webcam.sh; ui startup/isl-launcher.sh; desktop entry
Presentation	Tk window, widgets, key/button actions, preview	app/native_ui.py
Live orchestration	Camera capture, MediaPipe, scoring, temporal voting	app/webcam.py; app/native_ui.py
Inference contracts	Feature vector, count gate, confidence/margin rules	inference/landmarks.py; app/webcam.py
Model	26-way logits from 134 features	models/landmark_mlp.py
Training data	Manifest samples, MediaPipe extraction/cache	data/prepare.py; data/landmark_datasets.py
Training/evaluation	Optimization, checkpoints, reports	training/common.py; evaluation/common.py

Core boundaries

• Dataset and model locations are resolved centrally by config.py and can be overridden through environment variables.

• The manifest is the index for reproducible train/validation/test membership; image files remain on disk and are not copied into each split.

• Landmark extraction is shared in concept but has separate offline and live callers. Both must preserve feature ordering and normalization.

• The UI owns presentation and text editing. TextBuffer is independent of Tk and can be tested without a camera.

• The prediction result distinguishes an accepted label from a merely leading candidate; low-confidence or ambiguous frames should not be typed.

2. Repository map

Path	Purpose
README.md	Setup, workflows and project usage notes.
pyproject.toml	Python package metadata, Python version range, install-time console scripts and pytest configuration.
requirements*.txt	Base dependency list and CPU/CUDA-specific PyTorch installation options.
config/default.json	Default image size, seed, batch size, epoch count, confidence/margin and temporal controls.
src/isl_recognition/	Installable package source.
src/isl_recognition/app/	Webcam CLI/orchestration and native Tk UI.
src/isl_recognition/data/	Dataset acquisition, validation, split manifest, image and landmark datasets.
src/isl_recognition/evaluation/	Checkpoint loading and evaluation reports.
src/isl_recognition/inference/	Image inference, hand feature construction, background gate and smoke check.
src/isl_recognition/models/	Landmark MLP and legacy static CNN.
src/isl_recognition/training/	Static landmark training command and reusable training loop.
tests/	Unit and integrity tests.
artifacts/	Model checkpoints and caches; local model files are generally ignored by Git.
data/	Dataset provenance plus locally supplied/raw data outside the tracked code.
reports/	Generated evaluation outputs when present.
output/pdf/	Human-readable PDF deliverables, including this reference.
ui startup/isl-launcher.sh	Menu launcher for training/webcam and camera index selection.
launch-isl-webcam.sh	Direct webcam convenience launcher.
Launch ISL Webcam.desktop	Desktop-environment shortcut.

Repository state caveat: paths listed as generated or ignored may differ on a fresh clone. The README and source are the authoritative instructions for obtaining local data and creating artifacts. The raw dataset is not embedded in this guide or necessarily version-controlled.

3. End-to-end workflows

3.1 Dataset to trained model

Download (optional) → validate → build manifest → extract MediaPipe features/cache → train → select best validation checkpoint → export deployment checkpoint. Every stage is a distinct CLI entry point. Training data is filtered to examples where hands were detected; the feature cache is associated with a manifest signature so a stale cache is not silently treated as current.

3.2 Live webcam recognition

Launcher chooses a camera index → webcam command parses configuration → native UI initializes Tk on the main thread → model and MediaPipe tracker load → camera frames are captured → tracker returns hand landmarks → landmarks become a 134-value feature vector → MLP produces scores → hand-count compatibility masks unsupported classes → confidence/margin checks yield accepted/unaccepted candidate → a rolling frame vote stabilizes prediction → the UI displays the status and current sign → manual Add Letter or 500 ms auto-type appends to the text buffer.

3.3 Still-image inference

isl-infer-image opens one supplied image, loads the requested checkpoint, chooses a model-compatible input route, and prints a JSON result. Landmark checkpoints require MediaPipe extraction; no detected hands yields a no-hand result. Legacy CNN checkpoints use pixel preprocessing. The image route is useful for a single example, not a substitute for live temporal confirmation.

3.4 Evaluation

The evaluator loads the test split from the manifest, selects landmark or image dataset according to checkpoint architecture, predicts and applies hand-count compatibility where applicable, then writes machine-readable and text classification reports, a confusion matrix image, latency and coverage metrics. Accuracy among detected examples must be distinguished from coverage-adjusted accuracy over all original test samples.

4. Data lifecycle and split integrity

4.1 Acquisition and provenance

data/download.py defines the source dataset slug/version, invokes Kaggle CLI, downloads an archive into a temporary location, calculates SHA-256, safely extracts it, and records provenance. It refuses to overlay a non-empty target. Archive extraction checks destination paths so a malicious ZIP entry cannot write outside the destination directory. Kaggle authentication and network availability are environmental prerequisites.

4.2 Dataset discovery

data/common.py locates an image root by checking the expected original_images folder and the dataset root, then maps class folders case-insensitively to the requested labels. This centralizes assumptions about folder names and supported image suffixes. Ambiguous duplicate folders for a class are rejected rather than silently combined.

4.3 Validation

data/validate.py inspects class folders and files. It attempts to decode image files, records unreadable files, counts samples by class, hashes image bytes, and reports exact duplicates. Exact duplicate content across different labels is a conflict because it makes supervised labels contradictory. The CLI writes a validation report and returns failure when the dataset does not meet requirements. Validation is a data-quality gate, not a recognition score.

4.4 Manifest and near-duplicate grouping

data/prepare.py computes a 64-bit difference hash (dHash) per image after reducing it to a grayscale comparison grid. Within each class, samples whose hashes are within Hamming distance 3 are joined into visual groups using UnionFind. Groups, rather than individual images, are assigned to train/validation/test so near-identical frames do not leak across splits. Splitting is deterministic from the seed plus class label and targets 70/15/15 proportions. At least five distinct groups are required per class to make all three partitions viable. The manifest records paths, labels, splits, hashes/group membership, preprocessing information and source provenance.

4.5 Limits of visual grouping

The dataset has no source signer/session identifiers. The preparation note correctly warns that only visual near-duplicate grouping is possible. Therefore two images of the same signer or recording session can land in different partitions when they are not visually close. Reported validation/test performance can consequently overestimate performance on genuinely new people, camera setups, or backgrounds. Group-aware split is better than a random image split, but it is not signer-independent evaluation.

4.6 Image and landmark datasets

StaticImageDataset reads a selected manifest split, opens each image, applies shared RGB resizing/padding and returns a tensor plus class index. StaticLandmarkDataset uses a cached or newly extracted feature matrix, drops samples for which no hand was found, retains label/path records for reporting, and tracks per-class training counts by detected hand count. A sample can exist in the manifest but be absent from the effective landmark dataset because MediaPipe did not find a hand.

5. Landmark representation

A hand contributes 67 numbers. The first 63 are 21 three-dimensional landmark coordinates expressed relative to that hand’s wrist and normalized by hand scale. Four metadata values describe the hand center x, center y, relative scale, and presence. The two hand slots are sorted by horizontal center to make ordering consistent across frames; the vectors are concatenated into 134 input features. A missing second hand occupies the second slot as absent/zero values. If no hand is detected, feature construction returns None rather than a fabricated all-zero example.

Field	Meaning	Purpose
x/y/z × 21	Wrist-relative, scale-normalized landmark coordinates	Retain finger pose and geometry while reducing translation and size sensitivity.
center x, center y	Normalized position in frame	Retain coarse placement/arrangement cues; coordinates are bounded to a normalized range.
hand scale	Hand extent relative to smaller frame dimension	Keep relative hand-size cue after coordinate normalization.
presence	Binary slot marker	Distinguish a real hand from the padded absent slot.
two ordered slots	Left-to-right center order	Stable concatenation for one-hand and two-hand signs.

Hand-count compatibility gate

The checkpoint stores counts of training examples for each class at detected hand counts. At inference, classes are considered compatible with the current count only when at least 25% of that class's recorded training samples used that count. Incompatible class scores are suppressed and the remaining scores are renormalized. If required metadata is missing or no class supports the observed count, the helper returns the unmodified scores rather than creating an empty distribution. This is a data-derived class mask, not a second classifier; it does not guarantee that a sign was detected correctly.

Why the exact contract matters

Feature ordering, normalization, hand sorting, padding, metadata fields, and the feature-schema version must match during extraction, training, evaluation, and live inference. A shape of 134 alone does not prove semantic compatibility. Checkpoint metadata and feature schema should be updated together whenever this representation changes.

6. Models and preprocessing

6.1 LandmarkSignMLP

The primary model accepts a batch shaped [batch, 134] and emits [batch, 26] logits in A–Z label order. Its layers are Linear(134, 256), LayerNorm, ReLU, Dropout(0.30), Linear(256, 128), LayerNorm, ReLU, Dropout(0.20), Linear(128, 64), ReLU, then Linear(64, 26). The output is raw logits; softmax is applied by the caller. Layer normalization is independent of batch statistics and fits compact feature vectors. Dropout regularizes the dense model during training and disables itself in evaluation mode.

6.2 Legacy StaticSignCNN

The legacy CNN accepts three-channel square images and emits class logits. It uses convolution blocks with 16, 32, 64 and 96 channels, batch normalization, ReLU and max pooling, followed by adaptive pooling, dropout and a fully connected classifier. It exists for older checkpoints and compatibility tests. The live native UI explicitly requires architecture static_landmark_mlp, so a CNN checkpoint cannot currently drive that UI. Image inference/evaluation keep compatibility for old image checkpoints.

6.3 Pixel preprocessing

preprocess_image converts an image to RGB, preserves aspect ratio, resizes bilinearly to fit within the configured square, centers the resized image on a black square, converts pixels to float32 in [0,1], and returns channels-first layout. preprocess_path opens a path and delegates to the same implementation. This shared transform prevents train/inference drift for the CNN route. The default square size is 96.

6.4 Labels

labels.py declares ALL_LABELS as A–Z plus digits and STATIC_LABELS as A–Z. The current sign model is a 26-class alphabet model; the presence of digits in the source-label universe does not mean digits are trained or recognized. Class indices follow the label tuple order and must be kept aligned with checkpoint labels.

7. Training and checkpoint lifecycle

Training setup

train_static.main parses dataset/manifest/output paths, epoch count, batch size, seed and optional resume checkpoint. It builds train and validation landmark datasets, reports coverage and detected-hand counts, constructs LandmarkSignMLP with feature schema/normalization metadata, computes input-data provenance, then invokes the shared loop. Runtime depends heavily on CPU/GPU, the number of cached features, and whether MediaPipe extraction is already complete. Landmark extraction usually dominates first-run setup; later training reads cached arrays.

Shared training loop

Function	Behavior
set_seed(seed)	Seeds Python, NumPy, Torch and CUDA; requests deterministic algorithms; bounds Torch CPU threads.
class_weights(dataset, num_classes, device)	Counts train examples per class, rejects missing classes and returns inverse-frequency weights.
_evaluate(model, loader, criterion, device)	Runs inference without gradients and returns sample-weighted loss and accuracy.
_write_curves(history, output_dir, kind)	Writes accuracy and loss curve PNGs for train and validation history.
run_training(...)	Creates loaders/model device, weighted cross-entropy, AdamW optimizer, trains epochs, evaluates, saves latest/best checkpoints, early stops, exports deployment model.

Optimizer is AdamW with learning rate 0.001 and weight decay 0.0001. Loss is class-weighted cross entropy. Early stopping patience is six stale validation-loss epochs. The best checkpoint is chosen by validation loss; a higher validation accuracy on a later epoch does not necessarily replace it. Checkpoints contain model/optimizer state, progress/history, random-number states, labels, architecture, input configuration, provenance, class counts, environment versions and hand-count support counts. Resume checks the checkpoint kind, labels, image size, seed, batch size, provenance, counts and architecture-related metadata to avoid continuing a materially different run.

Checkpoint outputs

• _last.pt: latest resumable state.

• _best.pt: best validation-loss state.

• Deployment checkpoint (static_model.pt): compact inference payload with model weights and required metadata.

• Training history and plots: diagnostics; these do not alter the deployment model.

The checked-in or local artifact files are generated outputs, not the source of truth. Reproduce them from the dataset manifest and training command. Never rename an unrelated image model to static_model.pt: the UI validates architecture.

8. Prediction and evaluation

Prediction decision

For each frame, model logits are converted to scores. For landmark inference the hand-count mask is applied. The top score and the gap from the runner-up are compared with confidence and margin thresholds. If either threshold fails, the prediction is uncertain: the top candidate may still be shown for feedback, but an accepted/confirmed label is not produced. This distinction prevents every frame from becoming a typed letter. Exact result fields include status, accepted label, top candidate, score information and threshold context.

Image inference functions

Function	Responsibility
predict_image(...)	Load checkpoint, preprocess/extract appropriate input, run model, apply landmark count rule and confidence/margin checks, return structured result.
image.main()	Parse image/checkpoint/threshold arguments and print JSON prediction.
smoke_test.main()	Run a basic MLP shape/finite-output check on synthetic zero features; does not measure accuracy.

Evaluation functions

Function	Responsibility
load_model(checkpoint_path, expected_kind, device)	Load checkpoint metadata, choose MLP or legacy CNN architecture, restore weights, set evaluation mode and return model/checkpoint/device.
evaluate_checkpoint(...)	Score the specified test dataset; apply count gating; calculate classification metrics, confusion matrix, latency, detection coverage and coverage-adjusted accuracy; save JSON/text/PNG reports.
evaluate_static.main()	Parse paths, inspect architecture, select landmark or image dataset, call evaluator.

Interpret metrics carefully. “Accuracy among detected” ignores samples where the landmark extractor missed the hands. “Coverage” is the fraction of original test examples with usable landmarks. “Coverage-adjusted accuracy” combines recognition correctness with coverage and is the more conservative end-to-end figure. Per-class confusion reports are needed because aggregate accuracy can conceal letter pairs with similar hand shapes.

9. Live webcam and desktop interface

Window structure

UI region	Contents and behavior
Title bar area	ISL SIGN RECOGNITION heading and “Live alphabet fingerspelling” subtitle.
Camera preview	Large left pane. Displays live frame, landmarks and feedback; preview is mirrored for natural use.
Current sign	Large predicted letter or placeholder, plus status text.
Mode/status	STATIC A–Z and calibration/status state.
Confidence	Percentage plus progress bar; conveys score, not calibrated probability.
Hands detected	Current tracker hand count.
Recognized text	Text buffer shown in a fixed-height panel with line wrapping.
Auto-type status	Shows AUTO-TYPE ON/OFF and 500 ms hold behavior.
Controls	Add Letter, Space, Backspace, Clear, Auto-type, A–Z-only disabled indicator, Pause/Resume, Fullscreen.
Footer	Human-readable live status, e.g. calibration, no hand, stabilizing, uncertain.

Control behavior

Control	Mouse action	Keyboard	Effect
Add Letter	Click button	A	Append current stable accepted sign, subject to frame cooldown.
Space	Click	Space	Append a space; duplicate trailing spaces are prevented.
Backspace	Click	Backspace	Remove last character when present.
Clear	Click	C	Clear recognized text.
Auto-type	Click	T	Toggle dwell-based append; repeated held sign emits once until reset/new sign.
Pause / Resume	Click	P	Toggle camera recognition pause and clear temporal vote state.
Fullscreen	Click	F	Toggle Tk fullscreen.
Quit	—	Q / Escape	Destroy UI and trigger cleanup.

Native UI organization

native_ui.run(args, device) creates and initializes Tk on the main thread before loading Torch/MediaPipe. This ordering is intentional: it establishes the X11 connection before background inference/runtime threads start. It creates the window, theme and widgets, then maintains a state dictionary, TextBuffer, deque of recent frame predictions, and _DwellAutoTyper. Its nested functions format sections, refresh text, add letters, toggle modes, handle key events and process camera frames. Tk's root.after loop schedules frame updates without a separate GUI thread. Camera and MediaPipe resources are released on shutdown.

Temporal confirmation and duplication prevention

A deque stores the most recent predictions. A candidate becomes confirmed when it appears in at least 75% of the configured confirmation window (with a minimum count of two). Uncertain/no-hand frames do not become letters. Manual append also uses a frame cooldown. Auto-type adds only after the same candidate remains foreground-present for the dwell interval (currently 500 ms) and prevents a held sign from emitting continuously. Removing the hand or changing the candidate resets the dwell state.

Camera calibration and background gate

The application collects empty-scene frames to calibrate a simple background-difference gate. The gate compares absolute pixel changes to an average background and calculates foreground area. It helps decide whether meaningful foreground is present for auto-type/dwell behavior, but is not the hand detector. MediaPipe is responsible for hand landmarks. Calibration requires the camera view to be empty; motion, lighting changes and moving backgrounds can affect the gate.

10. File-by-file source reference: package and data

Package root and shared helpers

File / symbol	Detailed role
isl_recognition/__init__.py	Sets MEDIAPIPE_DISABLE_GPU=1 before the runtime import path. On Linux, attempts XInitThreads to prepare Xlib thread behavior; defines package version.
config.py · PROJECT_ROOT	Resolves repository/package root.
config.py · DATA_ROOT / STATIC_DATA_ROOT	Resolve data directories using ISL_DATA_DIR and ISL_STATIC_DATA_DIR overrides with repository defaults.
config.py · ARTIFACT_ROOT / CONFIG_PATH	Resolve model artifact root and config override (ISL_ARTIFACT_DIR / ISL_CONFIG).
config.py · load_config()	Read default JSON config and return a Python mapping for command defaults.
labels.py · ALL_LABELS / STATIC_LABELS / SOURCE_LABELS	Canonical class tuples; order is part of model serialization contract.
preprocessing.py · preprocessing_config(size)	Describe image size, color order, scaling and padding for checkpoint metadata.
preprocessing.py · preprocess_image(image, size)	RGB, aspect-preserving resize, black pad, [0,1] float normalization, CHW conversion.
preprocessing.py · preprocess_path(path, size)	Open a file and delegate to shared pixel preprocessing.
text_buffer.py · TextBuffer	Small state holder for recognized output, deliberately independent of UI toolkit.
TextBuffer.text	Read-only current buffer string.
TextBuffer.append_letter(letter)	Validate a single A–Z character and append uppercase form.
TextBuffer.append_space()	Append a space unless the buffer already ends in one.
TextBuffer.backspace()	Remove one trailing character safely.
TextBuffer.clear()	Reset to empty string.

Data package

File / symbol	Detailed role
data/common.py · IMAGE_SUFFIXES	Supported image filename suffixes.
locate_image_root(root)	Choose image directory based on expected dataset structure; fail clearly if absent.
image_paths_by_label(root, labels)	Return per-label paths; normalize folder casing and detect ambiguous folders.
data/download.py · _safe_extract(zip, destination)	Guard against ZIP path traversal before extraction.
download.main()	Obtain source dataset with Kaggle CLI, hash archive, extract, write provenance.
data/validate.py · inspect_dataset(root)	Build structured report of class counts, bad files, hashes and label conflicts.
validate.main()	Resolve dataset path, invoke inspection, persist report and exit according to validity.
data/prepare.py · UnionFind	Disjoint-set structure used to group visually near-duplicate images.
UnionFind.find(value)	Find representative with path compression.
UnionFind.union(left, right)	Merge sets by rank.
difference_hash(path)	Create compact grayscale dHash used for near-duplicate comparisons.
_split_groups(records, seed)	Deterministic group-level train/validation/test split; checks group-count sufficiency.
build_manifest(root, seed)	Validate inputs, group same-class near-duplicates, split and construct provenance-rich JSON.
prepare.main()	CLI wrapper that writes manifest/report.
data/visualize.py · main()	Generate a reproducible A–Z contact sheet for visual inspection.

11. File-by-file source reference: datasets, models and training

Dataset adapters and feature extraction

File / symbol	Detailed role
data/torch_datasets.py · StaticImageDataset	Index manifest rows for one split and return transformed image tensors and label indices.
StaticImageDataset.__len__()	Number of records in selected split.
StaticImageDataset.__getitem__(index)	Load image, preprocess with shared pixel transform, map label to index.
data/landmark_datasets.py · _initialize_worker()	Worker setup: limit OpenCV threading, instantiate MediaPipe tracker and arrange cleanup.
_extract_chunk(task)	Extract landmark feature rows for a path chunk in a worker.
load_or_extract_features(...)	Validate cache signature/shape; otherwise process chunks, report progress and save feature cache.
StaticLandmarkDataset.__init__(...)	Load manifest split plus features, discard no-hand rows, retain records and compute hand-count support.
StaticLandmarkDataset.__len__() / __getitem__()	Expose usable examples as feature tensor and class-index target.

Models

File / symbol	Detailed role
models/landmark_mlp.py · LandmarkSignMLP.__init__()	Declare architecture name, input feature count and dense layers for 26 classes.
LandmarkSignMLP.forward(features)	Pass batched landmark vectors through dense layers and return logits.
models/static_cnn.py · StaticSignCNN.__init__()	Build legacy convolution stack for square RGB images.
StaticSignCNN.forward(images)	Apply feature blocks, adaptive pooling, dropout and classifier.

Training

File / symbol	Detailed role
training/train_static.py · main()	CLI: parse dataset/training options, instantiate landmark datasets/model, collect provenance, launch common trainer.
training/common.py · set_seed()	Seed relevant RNGs and configure deterministic execution settings.
class_weights()	Compute inverse-frequency class weights; error if any expected class absent.
_evaluate()	Evaluate batches without gradient updates, aggregate loss and accuracy.
_write_curves()	Write train/validation accuracy and loss charts.
run_training()	Complete optimization/resume/checkpoint/export lifecycle, including early stopping and metadata consistency checks.

Although the current data objects share names like “static”, in this project “static” means a held pose or still-image alphabet sign. It is distinct from the old motion-sequence mode that was removed. The landmark model is still static-sign classification; it is not a temporal gesture recognizer.

12. File-by-file source reference: inference, evaluation and app

Inference and evaluation modules

File / symbol	Detailed role
inference/landmarks.py · hand_count_compatible_scores()	Mask/renormalize class probabilities using checkpoint per-class hand-count support.
landmarks_to_features()	Sort up to two hands spatially and encode 134-value input; return None when there is no hand.
extract_image_landmarks()	Read image, resize within safe dimensions, convert color and use a supplied tracker.
create_hands_tracker()	Create static-image MediaPipe Hands tracker for offline image extraction.
inference/background.py · BackgroundGate	Maintain background calibration and foreground area measurement.
BackgroundGate.update_background()	Add empty frame to calibration; signal when calibration is complete.
foreground_fraction() / has_foreground()	Measure thresholded changed-pixel fraction and compare configured bounds.
inference/predict.py · predict_image()	Run checkpoint-compatible image prediction and return structured status/candidate/scores.
inference/image.py · main()	CLI for one-image inference.
inference/smoke_test.py · main()	Fast architectural sanity check; synthetic input only.
evaluation/common.py · load_model()	Deserialize metadata and choose architecture/device; restore evaluation model.
evaluate_checkpoint()	Evaluate examples and persist text/JSON reports, confusion matrix and latency/coverage summary.
evaluation/evaluate_static.py · main()	CLI and architecture-aware dataset selection.

Webcam module helpers

File / symbol	Detailed role
app/webcam.py · _frame_tensor()	Convert OpenCV BGR frame to RGB and run legacy pixel preprocessor as a tensor.
_prediction()	Run model scores, optional count mask and confidence/margin acceptance logic.
_draw_hand_grid()	Render MediaPipe landmarks/connections for detected hands on the frame.
_wrap_sidebar_line()	Wrap text to a pixel width, including long unbroken strings.
_DwellAutoTyper.__init__()	Set dwell duration (default 0.5 seconds) and initial empty state.
_DwellAutoTyper.reset()	Clear tracked candidate/timing/emitted state.
_DwellAutoTyper.update(candidate, foreground_present, now)	Track stable foreground candidate and emit one letter after dwell; suppress repeat while held.
_compose_display(...)	Compose image-plus-sidebar OpenCV canvas, text, confidence, controls and landmark overlay.
app/webcam.py · main()	Parse camera/model/device/threshold/foreground options, initialize model/runtime and delegate to native UI.

Native UI nested callbacks

Nested function	Effect
section(label,row)	Place sidebar section heading.
update_text()	Copy TextBuffer content into Tk StringVar.
add_letter()	Append confirmed current letter unless missing or still in cooldown.
toggle_auto()	Flip auto-type flag, update UI label and reset dwell typer.
toggle_pause()	Flip paused state and clear temporal vote history.
toggle_fullscreen()	Set Tk fullscreen attribute.
on_key(event)	Dispatch supported keyboard shortcuts and return break to prevent duplicate widget handling.
update_camera()	Read frame, calibrate/background-check, track hands, encode features, predict, confirm, update widgets, render and schedule next frame.

13. Launchers, command-line surface and configuration

Console scripts

Command	Module entry	Use
isl-download-static	data.download:main	Fetch dataset and provenance.
isl-validate-static	data.validate:main	Check data files/classes/duplicates.
isl-preview-static	data.visualize:main	View A–Z dataset contact sheet.
isl-prepare-static	data.prepare:main	Create reproducible manifest/splits.
isl-train-static	training.train_static:main	Train landmark MLP.
isl-evaluate-static	evaluation.evaluate_static:main	Evaluate checkpoint on held-out split.
isl-infer-image	inference.image:main	Classify one still image.
isl-webcam	app.webcam:main	Start native live application.
isl-smoke-test	inference.smoke_test:main	Basic model execution sanity check.

Launcher behavior

ui startup/isl-launcher.sh presents a shell menu for training/webcam workflows and supports camera index selection. The webcam selection can be entered as an argument or ISL_CAMERA environment variable; index 0 often maps to the integrated camera on Linux but device numbering is system-specific. launch-isl-webcam.sh is the direct path and honors model/camera environment configuration. The .desktop entry exposes a desktop shortcut and uses a fixed path, so moving the project may require updating its Exec path. Scripts depend on the virtual environment and installed package being available as expected by the script.

Default configuration

Key	Default	Meaning
image_size	96	Legacy image input square size and recorded preprocessing metadata.
seed	42	Deterministic data/training seed.
static_batch_size	16	Training batch size.
static_epochs	40	Maximum epoch count; early stopping may stop sooner.
confidence_threshold	0.75	Minimum top score for accepting a prediction.
margin_threshold	0.15	Minimum gap between top two scores.
confirmation_frames	8	Rolling live frame window used for stable sign confirmation.
cooldown_frames	12	Minimum frame gap between manual letter appends.

Command-specific flags and environment-variable overrides are defined in each parser and launcher. Inspect --help on the installed command when exact accepted spellings matter; this PDF intentionally avoids asserting options that may have changed in an uncommitted local edit.

14. Tests and what they prove

Test file / case	Coverage	What it does not establish
test_labels.py	A–Z static label coverage.	Does not measure model accuracy.
test_models.py	Legacy CNN input/output shape.	Does not prove the installed deployment checkpoint matches this model.
test_preprocessing.py	Image and path preprocessing consistency, invalid-size rejection, metadata description.	Does not assess hand landmark inference.
test_split_integrity.py	Near-duplicate groups remain isolated across splits and labels appear in partitions.	Cannot guarantee signer/session separation; source IDs are absent.
test_text_buffer.py	Append, space, backspace, clear and invalid-letter behavior.	Does not test Tk widget wiring.
test_validation.py	Readable counts and duplicate/label-conflict validation.	Does not verify external dataset provenance.
test_webcam_ui.py	Canvas/sidebar layout utilities, wrapping, fullscreen scaling, dwell behavior, both-hand drawing.	Does not start a real camera or verify X11/Tk/MediaPipe on the host.

Known code/test discrepancies seen during source inspection

One known discrepancy remains in the checked-in test source: test_webcam_ui names a one-second dwell expectation, whereas _DwellAutoTyper defaults to 0.5 seconds and the UI text says 500 ms. The pause callback’s stale reference to an undefined clip_frames variable was removed: pausing now clears the rolling votes and current candidate, resets dwell auto-typing, clears the score and updates status. Tests were not run as part of producing this reference.

A further review target: test_webcam_ui’s helper tests cover `_compose_display`, while the live UI is implemented with Tk in native_ui.py. Keep any retained OpenCV canvas helpers clearly identified as utility/legacy rendering so tests do not give a false impression that they exercise the active Tk interface.

15. Operational limitations and interpretation

• Recognition quality is bounded by dataset diversity, label quality, MediaPipe detection coverage and overlap between similar poses.

• Current report numbers recorded with the repository artifacts: best validation loss at epoch 11; highest validation accuracy 83.78% at epoch 15; held-out set 3,370 source samples, with landmarks for 2,430 (72.11% coverage);

count-gated accuracy among detected examples 76.38%; coverage-adjusted accuracy 55.07%; reported MLP-only CPU latency 0.16 ms. These figures are artifact/repository context, not a guarantee for a different model file or camera.

• The discrepancy between highest validation accuracy and best-loss checkpoint is expected because training exports by validation loss.

• Camera index numbers vary by kernel, desktop session, permissions and attached devices. A valid camera may not be index 1; a USB webcam can appear under another index or by path.

• Background calibration assumes a relatively empty, stable scene for the initial sample window.

• Confidence scores are softmax-derived model scores; they are not necessarily calibrated real-world probabilities.

• There is no motion-sign mode in the current product flow. Landmark inputs are static frames; temporal logic is for stabilization and typing, not learning a motion sequence.

• The included UI and launcher target Linux; X11/Wayland and Tk availability can affect startup. Camera permissions and OpenCV backend availability are host concerns.

Security and data handling

Dataset extraction includes a path traversal check. Dataset download relies on the Kaggle CLI and user credentials outside this package. Checkpoints are loaded with PyTorch serialization, which is intended for trusted project-produced files; do not load untrusted checkpoint files. Raw images may contain people and should be handled as personal data by the project owner. Provenance hashes identify content but do not independently prove licensing or consent.

16. Maintenance guide and extension points

Adding or changing labels

Update the canonical tuples, dataset folders, manifest validation, output class count, training metadata and tests together. Existing checkpoints serialize their exact label order and are incompatible with a reordered class list. A model trained with fewer classes must not be interpreted through a 26-class UI.

Changing feature geometry

Update the feature schema/version, input dimension and documentation in landmarks.py; regenerate feature caches; rebuild train/validation/test dataset objects; retrain; update checkpoint loading and tests; then evaluate coverage as well as accuracy. Cache signatures must invalidate old features. Never mix features produced by two schemas.

Changing hand-count gating

Treat support threshold, metadata counts and score normalization as part of inference behavior. Keep evaluation and webcam callers aligned. Compare per-class accuracy and coverage for each hand count; an aggressive gate may prevent plausible classes from being chosen, while a permissive gate may restore confusing classes.

Changing UI behavior

Keep recognition decisions in testable helper functions where practical; keep Tk callbacks small; ensure pause/reset/close paths clear only defined state; update visible button labels and keyboard help together. Any camera-loop refactor must preserve Tk main-thread widget updates and release VideoCapture/MediaPipe resources when the window closes.

Reproducibility checklist

• Record dataset source/version/hash and manifest hash.

• Keep the same label ordering and feature schema.

• Use grouped splits and record seed.

• Save best and last checkpoints with environment metadata.

• Evaluate only the untouched test split after model selection.

• Report both recognition accuracy and landmark coverage.

• Document camera, lighting, distance and hand-count conditions for manual checks.

17. Glossary and source inventory

Term	Meaning in this project
dHash	Perceptual grayscale difference hash for visual near-duplicate grouping.
Landmark	MediaPipe hand joint point; 21 points are used per detected hand.
Logit	Unnormalized model score before softmax.
Margin	Difference between top and second class scores used as ambiguity rejection.
Coverage	Fraction of samples where the landmark detector produced usable features.
Coverage-adjusted accuracy	Accuracy over the full source sample count, accounting for detection misses.
Manifest	JSON index of dataset paths, labels, groups, splits and provenance.
Dwell	Minimum continuous stable candidate time required before auto-typing.
Confirmation window	Recent webcam frames used to decide whether a prediction is stable.
Hand-count support	Training evidence describing how often a class appears with one/two hands.

Complete source inventory

Python package modules: __init__.py; config.py; labels.py; preprocessing.py; text_buffer.py; app/__init__.py; app/native_ui.py; app/webcam.py; data/__init__.py; data/common.py; data/download.py; data/landmark_datasets.py; data/prepare.py; data/torch_datasets.py; data/validate.py; data/visualize.py; evaluation/__init__.py; evaluation/common.py; evaluation/evaluate_static.py; inference/__init__.py; inference/background.py; inference/image.py; inference/landmarks.py; inference/predict.py; inference/smoke_test.py; models/__init__.py; models/landmark_mlp.py; models/static_cnn.py; training/__init__.py; training/common.py; training/train_static.py.

Tests: test_labels.py, test_models.py, test_preprocessing.py, test_split_integrity.py, test_text_buffer.py, test_validation.py, test_webcam_ui.py. Root operational/configuration files: README.md, pyproject.toml, requirements.txt, requirements-torch-cpu.txt, requirements-torch-cu124.txt, config/default.json, launch-isl-webcam.sh, ui startup/isl-launcher.sh, Launch ISL Webcam.desktop. Data, checkpoints, reports and PDFs are runtime/generated content and may not be tracked in a clean clone.

Closing summary

The system is a compact, modular static-sign research prototype: image files are validated and split by visual similarity; MediaPipe converts usable images into a fixed landmark representation; a dense PyTorch model classifies letters; count-aware filtering, confidence/margin rejection and temporal confirmation make live predictions safer to type; a Tk desktop interface provides camera preview and text controls. Its strongest engineering contracts are the manifest split, 134-value feature schema, checkpoint metadata, and separation between candidate predictions and accepted text. The pause callback clears the rolling prediction state and resets auto-typing; one stale dwell-test expectation remains to be reconciled with the 500 ms product behavior.